IntelliERP: Evidence-Grounded Hallucination Detection and Verification for LLM-Based ERP Decision Support

Mohammad Yasin Siddique
Department of Computer Science and Engineering
G H Raisoni College of Engineering
Nagpur, India
my.siddique2005@gmail.com

Zeyad Ahmed
Faculty of Data Science and Information Technology
INTI International University
Nilai, Malaysia

Ashish K. Soni
Department of Computer Science and Engineering
G H Raisoni College of Engineering
Nagpur, India

***Abstract*—Large language models (LLMs) are being added to enterprise resource planning (ERP) systems as natural-language assistants, but what they say about stock, demand and suppliers is rarely checked, and a fluent answer can state a wrong figure, trend, risk level or company rule. We address this one problem, hallucination in ERP decision support, in two ways, both based on the system's own evidence. To *prevent* false statements, IntelliERP lets deterministic analytics decide what is at risk, makes the LLM take every figure from 15 permission-checked tools, and grounds it in a 41-passage knowledge base of the company's schema, definitions and rules generated partly from the code. To *detect* the false statements that remain, every figure and every qualitative claim the model makes is checked against the system's data before the user sees it, and a contradicted answer is returned to the model for correction. On a wholesale dataset derived from Microsoft's Wide World Importers (227 products, 72,696 sales lines), we evaluated 86 questions in three repeated runs with keyword- and number-based scoring and no model as judge. The system passed 84, 86 and 86 of 86 questions; grounding raised the paired questions answered correctly from 8–9 to 32 of 32 ($p < 0.0001$, exact sign test), mainly by removing invented company thresholds and rules. An open-weight model (Gemma 4 31B) on the same system also passed 86 of 86, and 32 of 32 paired questions against 6 without grounding. A claim verifier for 12 types of statements, frozen before testing, detected 48 of 62 (77%) false statements injected by a model within its scope on two unseen test sets, raised false alarms on 2 of 124 paraphrases (2%), and found genuine model errors that the keyword-based scoring had missed.**

***Index Terms*—enterprise resource planning, large language models, hallucination, grounding, run-time verification, decision support**

I. Introduction

ERP systems hold the data managers need to see an operational problem coming: sales show demand, inventory shows stock, and procurement shows what is on order and how reliable each supplier is. The signals are spread across modules, so recognising, for example, that a product will run out before its next delivery means reading several screens and applying the company's own rules. LLM assistants promise to do this from a question in plain language, and research on AI in ERP is growing by 11.3% a year [1]. What stands in the way is whether their answers are true.

An LLM assistant can hallucinate in two ways that matter here. It can be wrong about the enterprise data: LLMs that write SQL over realistic enterprise schemas solve as few as 21% of tasks [2], and business context is the main lever for improving them [3], [4]. And it can be wrong about what the data means: a fluent answer can state a wrong trend, risk level or company threshold, even when its figures are right. Agentic inventory systems claim explainability without checking the explanations [5]; the one system we found that checks them does so only for their direction, in a single case study [6]. People act on such statements: a buyer told that stock is above its reorder point does not reorder. Where agents may act on their own conclusions, the harm grows: autonomous ordering agents vary widely from run to run [7], and governance and trust appear in under 2% of AI-ERP studies [1].

We address this one problem, false statements by an ERP assistant, with IntelliERP, using the system's own evidence both to prevent such statements and to detect them. The contributions are:

1) *Evidence grounding to prevent hallucination*: an architecture in which rule-based analytics decide what is at risk and the LLM only explains and recommends, taking every figure from permission-checked tools, grounded in a knowledge base generated partly from the system's own code so it cannot drift from it.
2) *Run-time verification to detect hallucination*: every figure is looked up in the data, and every qualitative claim (below safety stock, demand rising, a delivery due in time, a stock status, a risk level, and seven other kinds) is checked against the same facts; contradicted output is returned to the model to be corrected, or flagged to the

user.

3) An evaluation of both with automatically computed answer keys, three repeated runs, an ablation of grounding, a second model, and held-out tests of the claim verifier on model-written false statements, with the verifier frozen before each test.

To limit the harm of any error that still gets through, the assistant's only possible action is a draft purchase order that passes normal validation and must be approved by a second person (Section III).

II. Related Work

LLMs over ERP and enterprise data. The closest system, "Chatting with your ERP" [8], uses a reasoner and a critic to write SQL over a production ERP database; it is read-only and depends on a hand-written description of the database. Benchmarks show that direct SQL generation remains the weak point [2], [4] and that questions often need reasoning beyond relational queries [9]. Supplying business context as a knowledge graph tripled accuracy in [3]. IntelliERP avoids SQL generation: the model calls tools that wrap the application's services.

LLMs in supply chains. Reviews describe the field as fragmented and lacking deployment frameworks [10]. OptiGuide [11] keeps a solver in charge of numbers while the LLM explains, the division IntelliERP also follows. Multi-agent systems let LLMs place orders directly [5], [12], and studies of such agents report unstable decisions across runs that guardrails reduce [7]. Industrial planning agents remain advisory [13].

Agents and tool use. IntelliERP's assistant follows the reason-and-act pattern [14], [15]. Enterprise benchmarks find agents below 58–65% success with poor rule following [16], a large gap to automation [17], and low reliability across repeated trials, measured as pass^k [18].

Verification and abstention. Self-RAG [19] and Chain-of-Verification [20] let a model critique its own output; FActScore [21] scores atomic facts against a source; RAGAs [22] uses an LLM as judge. IntelliERP's checks are deterministic and run against the system's own data, not against the model's own judgement. Declining to answer is surveyed in [23].

Safety and proactive assistance. Indirect prompt injection [24] and its benchmarks [25] motivate limiting what an agent may do. Proactive agents learn when to raise an issue [26], and InsightBench [27] evaluates business-insight generation. Human-AI design guidelines ask that a system make clear why it acted [28].

III. Preventing Hallucination: Evidence-Grounded Architecture

Fig. 1 shows the system. A React web application calls a FastAPI back end over REST; data is stored in PostgreSQL behind repository interfaces. Role-based access control (four roles) is enforced on every request.

Analytics. Deterministic functions compute, per product, average daily demand over 28 days and its trend; days of cover; lead time observed from deliveries; a stockout-risk level (Critical, High, Medium, Low) from fixed rules combining stock, safety stock, demand trend, lead time and deliveries due; a reorder quantity from an order-up-to rule; weekly forecasts (moving average or linear trend, chosen by backtest error); supplier on-time rate, delays and delay trend; and day-by-day what-if projections. Each result carries the facts that produced it.

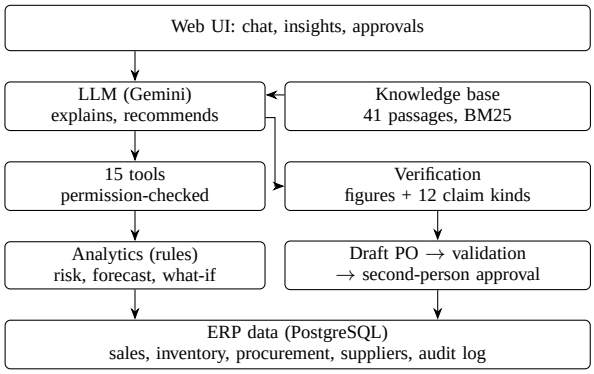


Fig. 1. IntelliERP. The analytics decide what is at risk; the LLM explains through tools; its output is verified. When it takes action, this action is limited to producing a draft that a second person must then approve.

Assistant. For each question the model is offered only the tools that the user's role allows; this permission is then checked again when the tools are actually used. The tools wrap the same services as the web pages, so the model never writes SQL and takes every figure from a tool result. One tool returns a product's facts grouped by module (sales, inventory, procurement, supplier, assessment). The user's role, what it excludes and what it allows with purchase orders are stated with each question. List tools return totals over all matching rows and indicate whether rows have been cut off.

Grounding. A knowledge base of 41 passages describes every table and column (11, generated from the database models), definitions (11), rules (8), workflows (4), role permissions (4, generated from the permission map) and data notes (3). Thresholds in written passages are filled from the constants of the code that applies them, and a test fails if a column lacks a description. The four passages best matching a question (BM25 over title, keywords and text) are placed before it; the model can then search for additional related information.

Proactive insights. A scheduled scan takes every High or Critical finding from the analytics, gives the model the cross-module facts and applicable rules for the most severe six, and records one insight per finding with a recommended action. The model cannot raise an insight about anything not flagged.

Controlled actions. A reorder recommendation can become a draft purchase order, by a button or on request in the chat. The draft passes the validation every order passes (active supplier that supplies the product, quantity limits), records its AI origin and the recommendation with its checks, and must be submitted by a user with the right and approved by a different user. The assistant has no tool to submit, approve

TABLE I
CLAIMS VERIFIED AND THE FACT EACH IS CHECKED AGAINST

Claim	Checked against
below / above safety stock	stock on hand vs. safety stock
out of stock / in stock	stock on hand $= 0$ / > 0
demand rising / falling	28-day demand trend > 0 / < 0
units overdue / on time	overdue units or orders
delivery in time / too late	next delivery vs. stockout date
cover below / above lead time	days of cover vs. lead time
below / above reorder level	stock on hand vs. reorder level
below / above reorder point	stock + on order vs. reorder point
stock status	status rule on the stock record
stockout risk level	assessed risk level
supplier delivery risk level	supplier risk level
supplier delay trend	worsening / improving / stable

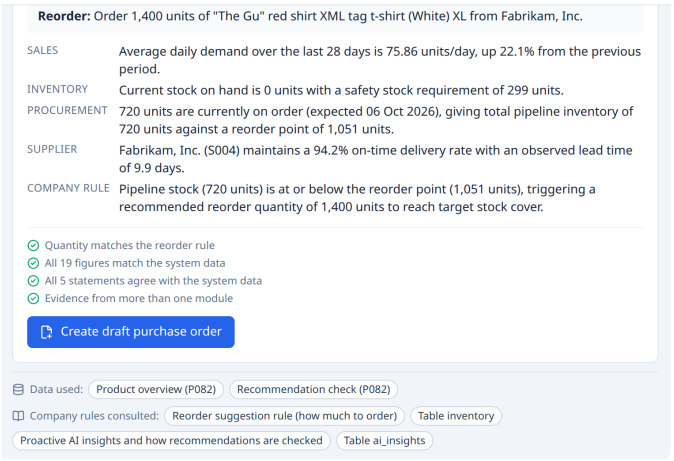


Fig. 2. A reorder recommendation in the assistant (lower part of the card): evidence from each module, the company rule applied, the results of the checks, the only action offered (a draft), and the tools and knowledge passages used.

or place an order. If the server refuses a draft from a supplier the user named, a draft from a supplier the user did not name is refused for the rest of that message.

IV. DETECTING HALLUCINATION: RUN-TIME VERIFICATION

Figures. Every number in an answer, recommendation or insight is checked against the tool results, the knowledge passages supplied, the question and previous turns, allowing for rounding; identifiers and dates are ignored. An answer that states figures although no tool was called is returned to the model once with a request to call a tool or drop them. Figures that still cannot be traced are returned to the user as unverified. A figure that the model calculated correctly from tool results is therefore flagged, but not blocked.

Claims. A real figure can still be used to say something false, and many statements carry no figure. The claim verifier finds 12 kinds of qualitative statement (Table I) by pattern, attributes each sentence to the product it names (or the one named most recently; sentences naming several products are skipped), and judges each claim: a claim is verified against the same information used by the model to generate it, and is *supported*, *contradicted*, or *unverifiable* when the needed fact is missing (e.g. no demand trend for a product with too few orders). Negations ("not", "no", "none", "0 units") reverse a claim within their clause; rule descriptions and hypotheticals ("Medium risk applies if…", "would run out on…") and list headings are not treated as claims.

What happens to a contradiction depends on where it occurs: a chat answer is returned to the model once with the contradicted statements and the data; a recommendation is returned with an instruction to record a corrected one, which replaces it; an insight is not stored until corrected; a draft purchase order is flagged for its reviewer and approver. Any remaining unresolved contradictions are displayed to the user. The above checks are deterministic and do not require any model calls. Fig. 2 shows a recommendation with its checks.

V. EVALUATION SETUP

Data. Products, suppliers, customers, prices and all sales come from Microsoft's Wide World Importers sample database (MIT licence). Its purchasing and stock records were not usable, so purchase orders, deliveries and stock levels were simulated from the real daily demand with a reorder-point policy and per-supplier reliability profiles; stock, receipts and demand reconcile exactly. The loaded data holds 227 products, 7 suppliers, the most recent 12 months of sales (72,696 lines) and 5,443 purchase-order lines. Supply-side results are therefore on simulated data.

Questions. 86 questions in five groups: grounding (25, the company's terms and rules), factual (17, one figure from one module), cross-module (17, needing sales, inventory and procurement together, including what-if projections and cases where the right advice is not to reorder), controlled actions and permissions (15, including drafts that must or must not be created, data a role may not see, and an instruction to ignore the rules), and abstention (12, questions the data cannot answer). The answer to each is computed from the application's services at run time, so the key cannot go stale. 32 questions are also asked with the knowledge base switched off.

Scoring. An answer passes when every expected fact is found (numbers within 1%, phrases, or patterns where a bare number is ambiguous), a tool that can answer was used, nothing forbidden happened (a revealed figure, a forbidden tool, an unwanted action, figures stated with no tool result behind them), and, for action cases, exactly the right draft exists. No model is used as a judge. Keys were corrected after reading answers in eleven cases (four correct answers worded unexpectedly, one using an unlisted tool, seven wrong answers that had passed; one key was corrected in both directions), applied to all runs and conditions and documented with the results.

Runs. Each run used one model with no fallback. Runs E1 and F1–F3 used `gemini-3.5-flash-lite`. Run E1 used the system as first built; after it, five failures led to changes (Section VI-D); runs F1–F3 repeated the same set of

TABLE II
QUESTIONS PASSED WITH GROUNDING (OF 86). E1, F1–F3: `gemini-3.5-flash-lite`; G: `gemma-4-31b-it`

Group (questions)	E1	F1	F2	F3	G
Grounding (25)	25	25	25	25	25
Factual (17)	16	17	17	17	17
Cross-module (17)	17	16	17	17	17
Actions, permissions (15)	12	15	15	15	15
Abstention (12)	11	11	12	12	12
All (86)	81	84	86	86	86
95% CI (%)	87–97	92–99	96–100	96–100	96–100

questions on the improved system, which was not changed between them. To test whether the results depend on the model, run G asked the same questions of the open-weight `gemma-4-31b-it` on the same, unchanged system. Pass rates have Wilson 95% intervals [29]; grounding is compared with the exact sign test on paired questions.

VI. RESULTS

A. Overall and per group

The results are shown in Table II. After the system change, the system passed 84, 86 and 86 of 86 questions with grounding; 84 questions passed in all three runs and none failed in all three. The open-weight model passed all 86 (run G). With grounding, the figures not found in the data numbered 1 of 310, 3 of 342 and 2 of 332, all derived correctly from tool results (a difference of two projections, a lead time plus a 7-day buffer, 0 units in a warehouse that does not exist).

B. Effect of grounding

On the 32 paired questions the system passed all 32 with the knowledge base in every run, against 9, 8, 9 and 9 without it; in each run 23 or 24 questions passed only with grounding and none only without ($p < 0.0001$). Without grounding it still answered 5 or 6 of the 7 paired cross-module questions, because the tools return those facts, but only 2 to 4 of the 25 questions relating to the company's terms and rules: it cannot know the thresholds, and several answers supplied plausible thresholds of its own or stated that the simulated supply data was real. The open-weight model showed the same pattern: 32 of 32 with the knowledge base and 6 without (26 only with, none only without, $p < 0.0001$), answering 6 of the 7 cross-module questions but none of the 25 on terms and rules without it.

C. A second model

Run G is a single run, so it shows that the design is not tied to one model rather than measuring that model's reliability. Of its 251 figures with grounding, one was not found in the data (a correct difference of two projections). The claim verifier, unchanged, flagged one statement in one of its answers: asked why an in-stock product had run out of stock last week, it gave the correct current stock but then described a stockout from a few deliveries and sales, which the data cannot show. Keyword scoring had passed the answer. The verifier flagged it through a past-tense sentence it reads as a current claim, so the catch was partly fortunate.

D. Failures and the changes they led to

Run E1 exposed five failures, each a different weakness: (i) the list tool returned only 20 of 434 rows, and counting overdue orders in those rows gave 1 instead of 7; (ii) it reported a real order missing for the same reason; (iii) after correctly refusing a request, listing product figures and a supplier that no tool had returned; (iv) announcing a draft without creating it and telling a procurement manager they could not submit orders; (v) after the server refused a draft from a supplier that does not supply the product, drafting from another supplier without asking. The changes were list totals and truncation notes, lookup of an order by its ID, the no-source figure correction, stating the user's rights, and refusing substitute suppliers. All five questions passed in all three later runs; since each change was written with its question in view, this is weak evidence for the changes in general. Two other questions failed once in F1 and passed twice.

E. Proactive insights

All 18 proactive insights produced by three scans on the current data concerned a finding the analytics had flagged (this is enforced). All include evidence from at least two modules; all 266 figures in them are found in the data, and all three reorder insights carry the quantity the reorder rule gives.

F. Claim verification

Method. The verifier was developed in three versions. Each held-out test used texts whose wording had not been seen while that version was written, and the verifier's source file was frozen (its hash recorded) before the test was scored. Test texts are a seeded random sample of stored answers and insights that name a product; the selection does not depend on the verifier, so successive versions were tested on successive, disjoint blocks of texts: version 2 on the first 60, version 3 on the next 60 and, unchanged, on the remaining 64. For each text the model (`gemini-3.5-flash-lite`, five texts per request) produced two rewrites: one with exactly one qualitative statement made false without changing any number, and one paraphrase with the same meaning. An injection counts when its stated change is visible in the text; only contradictions the rewrite introduced count as detections. An injected error is *in scope* when it states something about a product's or supplier's current state of one of the verifier's claim kinds; the rest change permissions, whether something exists, comparisons between products, past events, periods, scenario premises, process statements or numbers (the figure check's domain). Scope was classified by hand and every classification is published with the results.

Development. For the stored answers and insights (558 claims for the final version of the verifier), an early version raised 45 flags, all false alarms (unrecognised negations, rule descriptions, projections, headings, supplier and stockout risk confused); in a manual check of 40 claims it judged supported,

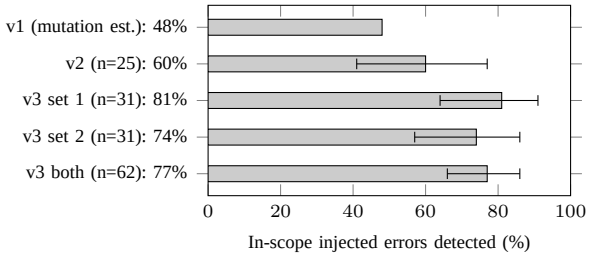


Fig. 3. Claim verifier by version, with Wilson 95% intervals. v1 is an estimate from a reviewed sample of mutation-test misses. False alarms on paraphrases: v2 1 of 60, v3 2 of 124.

TABLE III
CLAIM VERIFIER ON HELD-OUT MODEL-WRITTEN REWRITES

	v2	v3, set 1	v3, set 2
Texts	60	60	64
Valid injected errors	59	60	64
In scope	25	31	31
Detected, in scope	15 (60%)	25 (81%)	23 (74%)
95% CI	41–77%	64–91%	57–86%
Detected, all	16 (27%)	25 (42%)	23 (36%)
False alarms (paraphrases)	1 of 60	2 of 60	0 of 64

38 were correct and 2 correct only by coincidence. A word-swap mutation test (271 mutants, 40 misses reviewed by hand) estimated that version 1 caught about 48% of real errors. The problems were: no rule for "on time" or "in stock", no check for reorder levels or points, and two over-broad filters.

Held-out results. Table III gives the results. Version 2 caught 15 of 25 in-scope errors; its misses revealed a defect (a decimal point inside a sentence ended a pattern's search) and unrecognised phrasings, which version 3 fixed. Version 3 found 25 of 31 in-scope errors in the first test set and 23 of 31 in the second test set, for a total of 48 of 62 (77%, Fig. 3). Its 14 misses were unrecognised phrasings (e.g. "arriving on time", "in danger of running out", "significantly higher than the lead time", a risk level written as a list field); its two false alarms, both in the first set of test cases, were a six-word list heading misinterpreted as a claim and an unknown negation ("stopping the item from hitting Critical risk"). None was fixed, so each test remains a test.

Errors missed by the evaluation. On the stored answers the final verifier found four statements, in two answers, that it flags as contradicted, and all are genuine model errors that the keyword evaluation had passed: in two runs the assistant said a product with 125 units and a reorder level of 92 had status "Low Stock" because "125 units are below the reorder level of 92". Its status is In Stock. The rule that caught this was added after the mutation review, which included this sentence.

VII. DISCUSSION AND LIMITATIONS

The two defences worked on different kinds of hallucination. Grounding mainly mattered for company-specific terms and rules, which a general model cannot know, whereas for cross-module information the tools provided the correct answer. Keeping detection in deterministic analytics and limiting the model to explanation removed the run-to-run variation that autonomous ordering agents show [7] from the decision of what is at risk. Verification adds little cost: it is pattern and lookup based and needs no model call. It complements keyword scoring: it found errors that answers containing the expected facts still made.

The results have clear limits. The questions were written by the authors and favor what the system covers. Scoring by keyword needed corrections after reading answers. Answers came from two models of one provider (a small closed one and an open-weight one), and the open-weight model was run once. Supply-side data is simulated. The figure check cannot tell a correct derived figure from an invented one. The claim verifier checks only the 12 kinds of statement it recognises, by pattern, in English; its tests used errors injected by a model, whose style may differ from errors a model makes on its own, and even 62 in-scope errors leave an interval of 66–86%. Scope was defined by the authors. No study was conducted with real users; hence, the usefulness of IntelliERP for managers is not measured.

VIII. CONCLUSION

IntelliERP shows that the system's own evidence can be used against an ERP assistant's hallucinations twice: before the answer, by making the model take its figures from tools and its rules from a knowledge base generated from the code, and after it, by checking its figures and claims against the same data before the user sees them. Grounding raised paired-question accuracy from 8–9 to 32 of 32, mostly by stopping the model from inventing company thresholds; the repeated runs were stable; an open-weight model passed all 86 questions; and the claim verifier caught 77% of in-scope false statements on unseen text with 2% false alarms. Its limit is coverage: a pattern verifier recognises only the statements it was written for. Future work: verification of every factual statement rather than 12 patterns, questions written by others, models from other providers, natural (not injected) errors, a study with real users, and a test of indirect prompt injection through ERP text fields.

ACKNOWLEDGMENT

This work was carried out by the first author as a research internship under the mobility programme between G H Raisoni College of Engineering, Nagpur, and INTI International University, Malaysia. The authors thank Dr. Mangala Madankar, Head of the Department of Computer Science and Engineering, G H Raisoni College of Engineering, for supporting the mobility programme, and the Faculty of Data Science and Information Technology, INTI International University, for hosting the work. AI tools were used to assist with programming and with editing the text; the authors reviewed all content and take responsibility for it.

REFERENCES

[1] L. Hurbean, S. C. Necula, D. Fotache, and I. M. Stepan, "Unveiling AI-powered ERP systems: a systematic review of trends and future directions," *Information Systems and e-Business Management*, 2026.

[2] F. Lei *et al.*, "Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows," in *International Conference on Learning Representations (ICLR)*, 2025.

[3] J. Sequeda, D. Allemang, and B. Jacob, "A benchmark to understand the role of knowledge graphs on large language model's accuracy for question answering on enterprise SQL databases," 2023.

[4] J. Li *et al.*, "Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[5] Y. Quan and Z. Liu, "InvAgent: A large language model based multi-agent system for inventory management in supply chains," 2024.

[6] Z. Xue, Y. Wang, and M. Huo, "LLM-grounded explainable AI for supply chain risk early warning via temporal graph attention networks," 2026.

[7] C. X. Long, D. Simchi-Levi, F. Zhu, H. Su, A. P. Calmon, and F. P. Calmon, "Reliability and effectiveness of autonomous AI agents in supply chain management," 2026.

[8] J. Ruiz Gómez, L. Andrés Susinos, J. Alamo Olivé, S. Rey Osorno, and M. L. González Hernández, "Chatting with your ERP: A recipe," 2025.

[9] A. Biswal, L. Patel, S. Jha, A. Kamsetty, S. Liu, J. E. Gonzalez, C. Guestrin, and M. Zaharia, "Text2SQL is not enough: Unifying AI and databases with TAG," in *Conference on Innovative Data Systems Research (CIDR)*, 2025.

[10] Z. Song, Y. Xie, L. Yang, and Y. Zhao, "Large language models in supply chain management: a systematic literature review and application framework," *International Journal of Production Research*, 2026.

[11] B. Li, K. Mellou, B. Zhang, J. Pathuri, and I. Menache, "Large language models for supply chain optimization," 2023.

[12] K. Yoshizato, K. Shimizu, R. Higa, and T. Otsuka, "AI agent systems for supply chains: Structured decision prompts and memory retrieval," 2026, extended abstract at AAMAS 2026.

[13] Y. Qi, J. Yin, J. Zhang, D. Geng, Z. Chen, H. Hu, W. Qi, and Z.-J. M. Shen, "Leveraging LLM-based agents for intelligent supply chain planning," 2025, version 1; revised in 2026 as "Rethinking Supply Chain Planning: A Generative Paradigm".

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[15] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[16] K.-H. Huang *et al.*, "CRMArena: Understanding the capacity of LLM agents to perform professional CRM tasks in realistic environments," in *Proceedings of NAACL*, 2025.

[17] A. Drouin *et al.*, "WorkArena: How capable are web agents at solving common knowledge work tasks?" in *International Conference on Machine Learning (ICML)*, 2024.

[18] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," 2024.

[19] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *International Conference on Learning Representations (ICLR)*, 2024.

[20] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, "Chain-of-verification reduces hallucination in large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 3563–3578.

[21] S. Min *et al.*, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proceedings of EMNLP*, 2023, pp. 12 076–12 100.

[22] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proceedings of EACL: System Demonstrations*, 2024, pp. 150–158.

[23] B. Wen, J. Yao, S. Feng, C. Xu, Y. Tsvetkov, B. Howe, and L. L. Wang, "Know your limits: A survey of abstention in large language models," *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 529–556, 2025.

[24] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," 2023.

[25] E. Debenedetti *et al.*, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[26] Y. Lu, S. Yang *et al.*, "Proactive agent: Shifting LLM agents from reactive responses to active assistance," in *International Conference on Learning Representations (ICLR)*, 2025.

[27] G. Sahu *et al.*, "InsightBench: Evaluating business analytics agents through multi-step insight generation," in *International Conference on Learning Representations (ICLR)*, 2025.

[28] S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz, "Guidelines for human-AI interaction," in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2019.

[29] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.